Searching Code Patterns in Large Software Histories: An Incremental Query System for Structural Patterns and their Variants

Anonymous Author(s)

Abstract

Searching code is a fundamental aspect of source code analysis and every day development. Existing approaches are good at either searching abstract patterns in a single version, or textual patterns in source code histories. However, without precomputing diffs, to the best of our knowledge no approach provides both the expressivity of code patterns and the scalability needed to search entire code histories. To bridge this gap between expressivity and scalability, we propose an approach taking advantage of abstract code patterns without needing to write them manually. At its core, by indexing abstracted patterns on entire software histories and computing pattern inclusion, our approach enables searches to skip entire subtrees missing required patterns. Complementarily, to alleviate the tedious task of writing patterns, we present a technique to generate the abstract code patterns from concrete code examples, which itself leverages the search to classify and rank patterns, providing insights into the concrete variability of searched source code. In our evaluation we investigate patterns of test assertions on exceptions in Java, as the `try statement` variant became a bad smell when JUnit5 introduced their lambda expression alternative.

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords: software engineering, code search, indexation, performance computing, search engine, code patterns, pattern mining, formal concept analysis

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SLE'26, Rennes, France
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN TODO
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2018. Searching Code Patterns in Large Software Histories: An Incremental Query System for Structural Patterns and their Variants . https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large scale code search and navigation is an essential aspect of modern software development and research [9, 21, 23], from learning by example [5], to finding potential security vulnerabilities [1, 18]. With the rise of Version Control Systems (VCS) and hosting platforms, notably Git and GitHub, and ever-increasing collaborative and open-source development, it is now possible to search code in millions of repositories, some containing tens of millions of lines of code and over a billion commits.[1,2]

However, to obtain exhaustive results in reasonable time, searching a large amount of code poses fundamental challenges involving opposing objectives, namely scalability and expressivity [12, 26]. This opposition is especially relevant for search indexes where scalability relies on minimizing the cost of a search by minimizing memory accesses through a precomputed indexing structure reused across searches. However, building an index requires to choose its content, as well as, the way it is computed and persisted, in turn this restrains the expressivity of searches– that are effectively accelerated by the index. A typical indexing system for natural-language would index by words, or sequences of words, while maximizing selectivity of the index in the limit of available memory. GitHub notably first attempted using a natural-language indexed search[3], then later implemented their own code specific approach[1].

To enable code searches capable of expressing precise abstract tree structures, we adapt Tree-Sitter Queries [20] (TSQ), which were originally introduced in the Atom editor to configure syntax highlight. TSQ are symbolic expressions (sexpr) augmented by abstract and concrete types, quantifiers, captures, and roles, but limited to positive queries and explicit nested patterns (without transitive closure on the parent-children relation). Yet, we leverage the simplicity and limitations of TSQ to reason about code patterns, notably by testing pattern inclusion, synthesizing

[1] https://github.blog/news-insights/product-news/github-code-search-is-generally-available/
[2] https://archive.softwareheritage.org/browse
[3] https://github.blog/engineering/a-brief-history-of-code-search-at-github/

patterns from examples, and generalizing patterns into further abstract patterns. To scale, we index abstract code patterns on subtrees of code. To improve usability, we automatically generate abstract patterns from examples.

In our evaluation, we investigate test assertions on exceptions in Java; more specifically, the try statements containing a call to fail in its body and asserts the type of exception thrown, we call a *try fail catch*. The try fail catch pattern is especially interesting for two reasons: it showcases the expressivity and precision of TSQ; it is also a pattern which many developers and researcher have encountered, as it became a bad smell when JUnit5 introduced an alternative leveraging lambda expressions. To evaluate our approach we consider the three following research questions on 20 well-known Java repositories:

RQ1: To what extent does our approach effectively find instances of patterns and their variants in large software repositories?

RQ2: To what extent does the granularity of files, as opposed to the AST level, influence the selectivity of abstract patterns?

RQ3: How effectively can we generate abstract code patterns representative of a set of examples?

2 Motivating Example

Searching code is a major aspect of software development. The large amount of open source code available online combined with platforms providing search features (*e.g.,* GitHub) offers new opportunities to understand and select software systems, as well as learning from exploring source code [24]. However, this large amount of code is also what makes search challenging, notably due the number of results found, often too large to be considered exhaustively. To search a large amount of code effectively, queries need to be able to be refined iteratively, gradually increasing specificity, requiring a fast and precise query system. Consequently, to motivate our approach we consider a specific pattern of code, which requires precision and can occur thousands of times in a code base. In the remainder of this section, we gradually narrow and argue the relevance of this pattern.

Tests asserting exceptions are particularly interesting to study. They are notably related to Exception Handling and Mystery Guest test smells which are the most refactored kind of test smells [19, 24]. Tests asserting exceptions can mainly be written in three ways: 1) A `try fail catch` which we define as a `try statement` with a body containing a call to the tested method followed by a call to `fail` (*e.g.,* from JUnit) and at least one `catch` clause that sometimes contains an assertion over the caught exception. 2) An annotation on the test method optionally providing the expected exception type. 3) A call to assertThrows taking the exception as first parameter and a lambda expression containing the call to the tested method. `assertThrows` was introduced

```
1 try {
2   l.get(0);
3   fail("Expected E to be thrown");
4 } catch (e E) {
5   assertEquals(e.msg(), "empty list" );
6 }
```

(a) example of `try fail catch` in Java

```
1 E e = assertThrows(E.class,()->l.get(0));
2 assertEquals(e.msg(), "empty list" );
```

(b) the equivalent using `assertThrows`

```
1 (try_statement
2   (block
3     (statement_expression (method_invocation
4       name: (identifier)@fct (#eq? @fct "fail")
5     )))
6   (catch_clause))
```

(c) TSQ pattern of `try fail catch`

Figure 1. Try statement with a call to `fail` method

in JUnit 5 leveraging the lambda expression introduced in Java 8. Since then, researchers have interviewed developers and mined repositories to investigate test smells, finding that `assertThrows` were preferred over `try fail catch` [19, 24].

Figure 1a is an illustrative example of `try fail catch`, complementarily the equivalent assertThrows is presented in Figure 1b. While the `assertThrows` can accurately be found simply by searching for occurrences of `"assertThrows("`, instances of `try fail catch` can be more difficult to accurately find, notably with line-based textual searches. Indeed, three specificities make `try fail catch` hard to match: 1) a try statement can span over multiple lines, 2) `fail` is a less discriminating method name, 3) a call to fail could be in the try statement but outside the try body (*e.g.,* in the catch clause) which would not correspond to a `try fail catch`. Finally, using the Tree-Sitter Query syntax, Figure 1c presents a pattern matching `try fail catch` which follows the informal definition we provided earlier, on line 1 the try statement with a body (line 2) containing a call statement identified as `fail` (line 6) and a catch clause (line 10).

Figure 2 presents related and variant patterns of `try fail catch`, illustrating the kind of precision and control available through Tree-Sitter Queries. To facilitate the manipulation of patterns, we introduce positional predicates (Section 4.2), obtaining Figure 2a which is equivalent to Figure 1c (only differing at line 5 and 6). Figure 2b presents two patterns generalizing Figure 2a, while Figure 2c presents two specializations. The first generalization simply matches all try statements including the ones we could find outsize test code. The second generalization matches all calls to fail methods–which could as well be found in production code. Reversely the first specialization (line 1-10) specifies that the call to fail must be at the end of the try block (with the dot on line 8), while the second (line 11-20) requires that fail must be preceded by a statement (line 12)–which is

```
(try_statement
  (block
    (statement_expression (method_invocation
        name: (identifier) (#EQ? "fail")
    )))
  (catch_clause))
```

(a) initial pattern equivalent to Figure 1c

```
(try_statement)
(method_invocation
  name: (identifier) (#EQ? "fail"))
```

(b) two generalizations of a

```
(try_statement
  (block
    (statement_expression (method_invocation
        name: (identifier) (#EQ? "fail")
    )) .
  )
  (catch_clause))
(try_statement
  (block
    (statement)
    (statement_expression (method_invocation
        name: (identifier) (#EQ? "fail")
    )))
  (catch_clause))
```

(c) two specializations of a, respectively requiring `fail()` at the end of block and requiring at least a statement before `fail()`

Figure 2. Equivalent pattern using our positional predicates, along generalizations and specializations

an abstract syntax node. Enabling such a fine control over searched pattern is a major benefit of our approach.

3 Background

Our approach combines two state-of-the-art tools to process and analyze source code. First, Tree-Sitter Queries (Section 3.1) provide a way of defining and matching code patterns halfway between regular expressions (regex) and XML Path Language (XPath). Second, HyperAST (Section 3.2) enables the incremental preprocessing of source code histories along with a memory efficient representation.

3.1 Tree-Sitter Query system

We choose an existing query system based on s-expressions created for the Tree-Sitter (TS) parser generator, which we abbreviate interchangeably as TS Queries and TSQ. In the near term, TS Queries offers the opportunity of synergizing with approaches and tools already using TS [1, 6, 8, 11, 13]. Moreover, TS Queries are used to highlight code rendered on the GitHub website, as well as in multiple code editors, such as, VS Code, NeoVim, Helix, Zed.

TS Queries balance expressivity and ease of implementation by expressing code-specific properties while defining patterns positively and without arbitrary nested patterns[4]. The reference C implementation is architectured around a compiler and an executor sharing an automaton as the intermediate representation (IR). The compiler produces a non-deterministic finite automaton from s-expression queries. Then, the automaton is executed in a single traversal of the AST, maintaining a set of live states which simulates the non-determinism. However, to the best of our knowledge, no formalization of TS Query exists. Consequently, we introduce one ourselves covering the core aspects of the query syntax (Section 3.1.1) and tree traversal (Section 3.1.2), as well as an automaton formalization (Section 4) for the intermediate representation. We do not intend our formalization to be complete nor a reference as it would require a significant amount of effort, notably regarding edge cases subject to changes in the reference implementation.

3.1.1 A Minimalistic Tree Pattern Syntax.

To balance expressivity and ease of implementation, the grammar of TS Queries structures patterns as s-expressions, leveraging individual TS grammars to define language specific aspects[4]. Indeed, s-expressions are easy to parse and often used to represent trees; they were already used to test TS grammars, representing the serialized concrete syntax tree which should match the one resulting from parsing. For example, in Java, `"try {} finally {}"` would correspond to `(try_statement (block) (finally_clause (block))`, such that, `try_statement`, `block`, and `finally_clause` are the node types corresponding to names of grammar rules defined for Java with Tree-Sitter. Then, adding to s-expressions, the TSQ language is augmented to make it a query language and enhance its expressiveness. Of these augmentations, other than mirroring regular expressions in terms of alternations, quantifiers, and captures, three extend code-specific capabilities distilled from the grammar of the queried language: anonymous nodes represent keywords and delimiters corresponding to unnamed terminal rules, abstract node types represent abstract groups of syntax nodes corresponding to grammar rules that are unions of other rules, and fields represent roles of nodes in their parents corresponding to annotations in the grammar. Consequently, TS Queries can express both precise and abstract code patterns.

Definition 1 (TSQ Grammar)**.** *Given the initial ⟨query⟩ rule, three language-dependant terminal (⟨label⟩, ⟨keyword⟩ and ⟨field⟩), and two used defined identifiers (⟨identifier⟩ and ⟨pred⟩), we define the TS Query grammar as follows:*

[4] issues #448 and#444 of the tree-sitter/tree-sitter project

$$
\begin{aligned}
\langle query\rangle &::= \langle pattern\rangle* \\
\langle pattern\rangle &::= \text{`(`}\langle label\rangle\langle children\rangle\text{`)`} && \diamond \textit{named node} \\
&\mid \text{`"`}\langle keyword\rangle\text{`"`} && \diamond \textit{anonymous node} \\
&\mid \langle pattern\rangle\ \langle capture\rangle && \diamond \textit{capture} \\
&\mid \langle pattern\rangle\ \langle predicate\rangle && \diamond \textit{predicate} \\
&\mid \text{`[`}\ \langle pattern\rangle+\ \text{`]`} && \diamond \textit{alternation} \\
&\mid \text{`(`}\ \langle pattern\rangle+\ \text{`)`} && \diamond \textit{group} \\
&\mid \langle field\rangle\ \text{`:`}\ \langle pattern\rangle && \diamond \textit{field} \\
&\mid \text{`!`}\ \langle field\rangle && \diamond \textit{negated field} \\
&\mid P\ \text{`?`} \mid P\ \text{`*`} \mid P\ \text{`+`} && \diamond \textit{quantifiers} \\
\langle children\rangle &::= \epsilon \mid \langle pattern\rangle\ \langle children\rangle \mid \langle children1\rangle \\
\langle children1\rangle &::= \text{`.`}\ \langle pattern\rangle\ \langle children\rangle \mid \text{`.`} \\
\langle capture\rangle &::= \text{`@`}\ \langle identifier\rangle \\
\langle predicate\rangle &::= \text{`(\#`}\ \langle pred\rangle\ \text{`?`}\ \langle argument\rangle*\ \text{`)`} \\
\langle argument\rangle &::= \text{`"`}\ \langle string\rangle\ \text{`"`} \mid \langle capture\rangle
\end{aligned}
$$

named node defines the s-expression foundation of the TSQ grammar, where ⟨label⟩ represents the concrete or abstract name of nodes (corresponding to rules in the language's grammar). ⟨label⟩ has three special values shared between all parsed languages: '_' the wildcard (*i.e.,* matches any label), 'ERROR' for nodes marking a parsing error, and 'MISSING' for nodes encapsulating an inferred keyword that fixes a parsing error. Additionally, the label can be a concrete node type prefixed by an abstract type (separated by a forward slash /). ⟨anonymous node⟩ represents leafs (*i.e.,* terminal rules) that do not have a dedicated named rule in the language grammar and which are defined between quotation marks (*e.g.,* `"public"` or `"try"`). ⟨capture⟩ marks a pattern with ⟨identifier⟩ to enable future references to captured nodes. *predicate* filters patterns based on captured nodes, where ⟨pred⟩ refers to predicates provided externally, such as 'eq' to test if the text of a captured node matches a given string (*i.e.,* ⟨string⟩) or another captured node. *alternation* defines alternative patterns. *group* defines groups of patterns. *field* filters nodes based on their role in their parent, where ⟨field⟩ is the third language-dependent terminals. *negated field* also uses ⟨field⟩, this time to filter out nodes containing children with the specified role. *children* enables the definition of subpatterns matching siblings sequentially, but not necessarily consecutive. *children1* alternatively enables stricter positioning/sequencing using a dot operator ., which we call the immediate operator. Compared to regular expressions, depending on its position in a list of siblings, the immediate operator behaves like concatenation, ^, or $. Moreover, it differentiates named and anonymous nodes, such that when adjacent sibling patterns are named nodes, the immediate operator only considers named tree nodes, otherwise it also constrains anonymous nodes.

3.1.2 Tree Abstraction and Traversal.

Implementation choices made for Tree-Sitter have considerable consequences on performance and modeling of the query system. Their understanding will help highlight our own decisions. At its core the TS syntax tree is composed of subtrees that a) are not rooted, and b) do not contain labels of leaves (*e.g.,* identifiers, number literals). Both enable compact, often contiguous, memory layout and cache friendly exhaustive traversals. However, (a) requires an additional abstraction to effectively traverse the tree, and (b) makes access to labels more difficult. As further consequence of (b), labels are only accessed by predicates which can be costly to compute, thus delaying and reducing predicates' executions is usually beneficial.

To address (a) and make the query executor more concise an abstraction called tree cursor is provided by TS. This tree cursor provides means of navigating the tree and retrieving properties of nodes. By abstracting the details of the underlying tree structure, it also facilitates the interoperability of the query system, to the benefit of the alternative tree structure we choose (see Section 3.2). Consequently, we focus on the essential aspects of tree nodes exposed through this cursor abstraction.

Definition 2 (TS Tree)**.** *In accordance with the abstraction of the TS cursor, we model the nodes of a TS tree as root nodes uniquely identified by a list of offsets in $\mathbb{N}^*$ where the root is the empty list ϵ. For a given tree the type* (i.e., *its label) of a node is given by the function $t : \mathbb{N}^* \to \Sigma_t$ and the set of nodes is defined as $w \in dom(t)$. Consequently, the depth of a node is defined as $depth(w) = |w|$. To accommodate fields on eligible nodes, we define the partial function $f : \mathbb{N}^* \rightharpoonup \Sigma_f$ such that $dom(f) \subseteq dom(t)$. Elements of Σ_t can be further discriminated as keywords, or abstract node types. Both Σ_t and Σ_f are language dependent. We do not, however, define an accessor for the text of a node, instead we define the set of predicate functions $\mathbb{P}$ which can hold any predicate (we expect functional purity) taking a list of nodes, such that*

$$\mathbb{P} = \left\{ \left(P, P^{(\mathbb{N}^*)^*} \right) \middle| P^{(\mathbb{N}^*)^*} \in \{\top, \bot\} \right\} \tag{1}$$

To navigate in a tree we consider three partial functions capable of transforming offset lists:

$$
\begin{aligned}
\downarrow &: \quad \mathbb{N}^* \rightharpoonup \mathbb{N}^* && \textit{move to first left child} \\
\rightarrow &: \quad \mathbb{N}^* \rightharpoonup \mathbb{N}^* && \textit{move to next right sibling} \\
\uparrow &: \quad \mathbb{N}^* \rightharpoonup \mathbb{N}^* && \textit{move to parent}
\end{aligned}
$$

3.2 Temporal and Incremental AST Preprocessing

To index and search large source code histories, we propose to compress syntax nodes in a Direct Acyclic Graph (DAG) using the HyperAST approach [17]. Compared to Git, which can be seen as a file system deduplicating files without preprocessing, the HyperAST deduplicates syntax subtrees which are obtained after parsing files (stored in a Git repository). The HyperAST enables these subtrees to be augmented with results of user defined local computations. These local results also benefit from the deduplication of subtrees, consequently using less memory. Additionally, computing these local results after deduplication (*i.e.,* on new unique subtrees) leads to emergent incrementality. This approach enables complex analysis involving local computation to scale to large and long software histories.

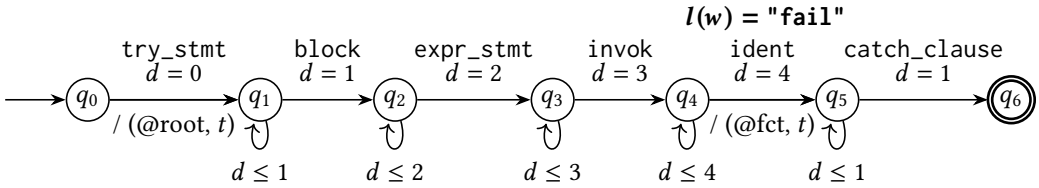


Figure 3. Automaton of `try fail catch` pattern with a positional predicate

4 Tree-Sitter Query Automaton

TS Queries are compiled into an Intermediate Representation (IR), which we call the TSQ IR. The TSQ IR represents the transitions of the Nondeterministic Finite Automaton (NFA)[4] interpreted in the core of the executor. The executor traverses the AST depth-first and simulates the nondeterminism by managing multiple states in parallel. When entering a tree node, a new state is spawned and states are updated; when exiting, states are deduplicated and terminated.

To precisely present our contributions, we finally chose to model the TSQ IR with Tree-Walking Automata (TWA). Indeed, for our approach which skips subtrees, TWA gives control of the tree traversal. We also found that TWA could precisely model the C implementation of Tree-Sitter, giving us room to easily model all features of TSQ. Even drawbacks of TWA [10] align with the limitations of TSQ, such as patterns being limited to explicit nesting, and nondeterminism being simulated (not determined).

Definition 3 (TSQ capture automaton). *We define the TSQ automaton as the (capturing) Tree-Walking Automaton* $A = \langle Q, \delta, I, F, \Gamma \rangle$ *where*

Q *is a set of states*
$\delta \subseteq Q \times \Phi(w) \times \{\uparrow, \downarrow, \rightarrow\} \times \Gamma \times 2^Q$ *is a set of transitions*
$I \subseteq Q$ *is a set of initial states*
$F \subseteq Q$ *is a set of final states*
Γ *is a set of captures (name and node)*
$\Phi_i(w)$ *is a predicate for each transition* i *over a tree* w

To later define how subpatterns are indexed and compared to patterns, we need to define a subtree automaton.

Definition 4 (Subtree Automaton). *The subtree closure is defined as a nested NFA* A' *guessing when to enter the nested NFA* A*, such that,*

$$\begin{aligned} \text{SubTree}(L(A)) &= L(A') \\ \textit{where} \quad A' &= \langle Q', \Sigma, \delta', q_I, F \rangle \\ Q' &= Q \cup \{ q_I \} \\ \delta' &= \delta \cup \left\{ q_I \xrightarrow{\epsilon} I,\ q_I \rightarrow q_I \right\} \end{aligned}$$

4.1 Immediate Operator Extension

To model the postional operator, particularly at the last position (*e.g.*, `(a (b) .)`), the TSQ transitions need to be extended, requiring the absence of right (named, if current matched node is named) siblings, with the predicates is_last_named(t) when t is named, is_last(t) otherwise. One could define such predicates by peeking into right siblings (using a 2-pebble TWA), however, due to this property being purely local and automata independent, the predicates can be considered as precomputed, and modeled by extending the input tree with the verdicts, such that:

$$\delta' \subseteq Q \times \Phi(w) \times \{\epsilon, Last, Last_{Named}\} \times \{\uparrow, \downarrow, \rightarrow\} \times \Gamma \times 2^Q$$

4.2 Positional predicates

We also introduce a new category of predicates that are position dependent and meant to be immediately executed when entering their targeted node, we call them positional predicates. In the context of our approach, positional predicates are orthogonal to our main contribution. However, reifying predicates in the automaton formalism facilitates the use of predicates in the precomputed patterns. Moreover, positional predicates do not use captures, simplifying transformations of patterns. Figure 2a presents a TS query using a positional predicate that is equivalent to the query in Figure 1c, which uses the capture-based predicate. Semantically, these immediate predicates take the form of additional conditions on transitions of the capture automaton in Figure 3. The same applies to the second subpattern in Figure 2b and the specialized patterns in Figure 2c.

5 Approach

Our approach aims to enable efficient and practical search of code patterns in large source code histories. To do so, we leverage synergies between two code representations, as illustrated in Figure 4. Bijectively, given a pattern, we can search tree nodes, and given tree nodes, we can generate patterns. To formalize and describe our approach, we use subpattern and subtree as specializations of pattern and tree with additional constraints. On the one hand, *subtree* directly follows the definition of the HyperAST approach. It represents a syntax node stored in a DAG. A DAG built from deduplicating structurally identical syntax node. On the other hand, *subpattern* represents a pattern in the context of the inclusion problem from automata theory. This is a central concept of our approach that we use pervasively, notably to test the inclusion of query in indexed patterns as well as to generate patterns and compare them.

Definition 5 (pattern inclusion). *Following the standard definition of inclusion in automata theory, a TSQ pattern* P *is included in another pattern* S*, denoted as* $P \subseteq S$*, when all trees accepted by* P *are also accepted by* S*,* $L(P) \subseteq L(S)$*.*

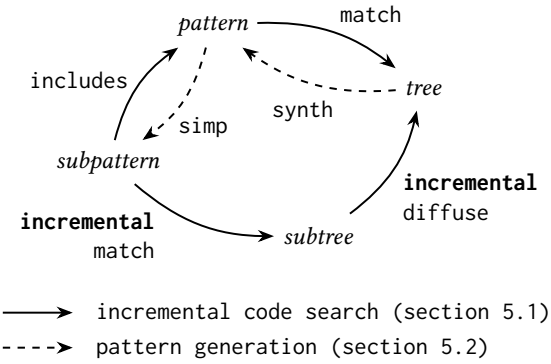


Figure 4. Overview of interactions between patterns, trees, and their sub-components in our approach.

We call S interchangeably abstract *pattern or* subpatttern. *Symmetrically, we call P a* specialized *pattern of S.*

Beyond the general interactions between patterns and syntax trees, we detail the synergies as they concretely apply to Tree-Sitter and HyperAST. The next section presents our incremental code search, which incrementally precomputes subpatterns, checks pattern inclusion, then skips subtrees. Reversely, Section 5.2 presents our pattern generation technique as well as a method to compare and rank generated patterns, leveraging the partial order defined through pattern inclusion and the code search itself.

5.1 Efficient Code Searching: Skipping Subtrees Missing Required Subpatterns

Under a naive search, given a pattern, an exhaustive tree traversal is required to locate all matching instances, even if there are none. To avoid unnecessary exhaustive traversals and spare memory accesses, we must skip subtrees that cannot match. Only considering simple constraints between patterns and syntax trees, we could consider skipping subtrees too short to contain the pattern, skipping those that do not contain mandatory node types, or 3) skipping those grammatically impossible in context, such as, a file inside a file or a nested function in ANSI C.However, these properties would have narrow application scopes, low selectivity, or depend on programming language peculiarities. Instead, we propose to index TSQ patterns themselves, leveraging pattern inclusion.

The problem of subpattern indexing can be defined by combining the pattern inclusion and subtree-closure, where $\mathbb{S}$ is a set of subpatterns required to match a pattern P, such that:

$$L(P) \subseteq \bigcup_{S \in \mathbb{S}} \text{SubTree}(L(S)) \tag{2}$$

Based on this formal description, we propose that our indexing be composed of two structures: the list of subpatterns $\mathbb{S}$ and the corresponding list of verdicts $\text{SubTree}(L(S))$ for each subtree. Considering them separately, we can represent lists of verdicts efficiently (*e.g.,* as bitsets), while the list of subpatterns is only used when compiling queries to find the offsets of verdicts.

5.1.1 Indexing Subpatterns.

To scale and leverage the incrementality of HyperAST, the verdicts of provided subpatterns are computed shallowly and bubbled-up to ancestors– without traversing again all the descendants for each subtree. More formally, let V the list of verdicts, be noted $V_S(s)$ for each HyperAST subtree s and each pattern $S \in \mathbb{S}$ such that,

$$V_S(s) = S(s) \vee \bigvee_{s' \in \mathit{children}(s)} V_S(s') \tag{3}$$

where $\mathit{children}(s)$ is the set of children of subtree s. For any given subtree, the query executor will only instantiate S on the root of the subtree s. This automaton can only reach a fixed depth d (*i.e.,* no arbitrary nesting). For the set $\mathbb{S}$, the traversal depth for indexing is the maximum depth between all S. Consequently, the indexing cost in numbers of reads is proportional to the largest d times the numbers of unique subtree.

Additional concerns impact the scalability of our indexing. On one hand, indexing subpatterns during the construction of a subtree maximizes efficiency by accessing already fetched memory. On the other hand, indexing additional subpatterns after the initial build offers flexibility but requires to read all subtrees and mutate the bitsets storing the verdicts; we call it late indexing. Considering the additional operations, this late indexing will always be less efficient than indexing during build. However, the drawbacks of late indexing can be mitigated to improve latency. It requires to organize subtrees by height, then indexing can be dome in parallel in each height group, sequentially increasing height.

The storage of verdicts is also important for performance. Probabilistic sets are particularly well suited; notably Bloom Filters which were already used in HyperAST to index reference signatures [17]. Here, the indexing of positive verdicts can be relaxed, such that additional subtrees might get traversed, although they will not produce matches, but no subtree containing a match will be skipped. Moreover, during the build phase we can choose the size of the Bloom Filter, such that we completely avoid saturation. Consequently, Bloom Filters should enable the indexing of a greater number of patterns within a reduced space in exchange for marginally reduced selectivity, without compromising the validity of searches.

5.1.2 Finding Required Subpatterns Through Pattern Inclusion.

Usually, $P \subseteq S$ is given by $\mathcal{L}(P) \cap \overline{\mathcal{L}(S)} = \emptyset$ such that we have to complement and union automata. Approaches exist to compute inclusion between different kinds of automata, Champavère *et al.* present one over tree automata and DTDs [7], Holík *et al.* present a complementation technique of NFA without full determinization [14]. However, inclusion can be costly to compute in the general

Input: set of subpatterns $\mathbb{S}$, set of patterns $\mathbb{P}$
Output:
set of mandatory subpatterns $\mathbb{M} = \{ \mathcal{S} \in \mathbb{S} \mid \mathcal{P} \in \mathbb{P}, \mathcal{P} \subseteq \mathcal{S} \}$

1 $Index \leftarrow \emptyset$
2 **foreach** $\mathcal{S} \in \mathbb{S}$ **do**
3 $h \leftarrow 0$; $q \leftarrow \mathcal{S}_0$
4 **while** $q \notin F_{\mathcal{S}}$ **do**
5 **if** $q \rightarrow q' \in \textsc{Sink}_{\mathcal{S}}$ **then** $q \leftarrow q'$
6 **else if** $q \rightarrow q' \in \delta_{\mathcal{S}} \backslash \textsc{Loop}_{\mathcal{S}}$ **then**
7 $q \leftarrow q'$
8 $h \leftarrow h + \textsc{Sig}(q \rightarrow q')$
9 $Index[h] \leftarrow \mathcal{S}$
10 $\mathbb{M} \leftarrow \mathbb{S}$
11 **foreach** $\mathcal{P} \in \mathbb{P}$ **do**
12 $q \leftarrow \mathcal{P}_0$; $stack \leftarrow \emptyset$
13 **while** $q \notin F_{\mathcal{S}}$ **do**
14 $stack.push((0, q))$
15 **if** $q \rightarrow q' \in \textsc{Sink}_{\mathcal{P}}$ **then** $q \leftarrow q'$
16 **else if** $q \rightarrow q' \in \delta_{\mathcal{P}} \backslash \textsc{Loop}_{\mathcal{P}}$ **then**
17 $q \leftarrow q'$
18 $\mathbb{N} \leftarrow \emptyset$
19 **while** $stack \neq \emptyset$ **do**
20 $(h, q) \leftarrow stack.pop()$
21 **if** $\mathcal{S} \in Index[h]$ **then** $\mathbb{N} \leftarrow \mathbb{N} \cup \{\mathcal{S}\}$
22 **if** $q \in F_{\mathcal{P}}$ **then continue**
23 **if** $q \rightarrow q' \in \textsc{Sink}_{\mathcal{P}}$ **then**
24 $stack.push((h, q'))$
25 **else if** $q \rightarrow q' \in \delta_{\mathcal{P}} \backslash \textsc{Loop}_{\mathcal{P}}$ **then**
26 $h' \leftarrow h + \textsc{Sig}(q \rightarrow q')$
27 $stack.push((h, q'))$
28 $\mathbb{M} \leftarrow \mathbb{M} \cap \mathbb{N}$

Algorithm 1: Pattern Inclusion Algorithm

case. We also modeled TS queries as nondeterministic tree-walking automata (TWA) which are difficult to manipulate and analyze, notably for inclusion. As an alternative, we restrained the subpatterns that our technique would consider, leveraging a more brute force enumeration and comparison of runs (*i.e.,* traces obtained by simulating the execution of a pattern). This approach should find simple inclusions, but due to its incompleteness it will miss some complex inclusions–notably those involving branches and cycles. But we assume a marginal impact on selectivity, as repetitions not encased in immediate operators only relax patterns and alternations only contain mandatory patterns when all alternatives have common runs.

Algorithm 1 presents our algorithm computing the inclusion between TSQ patterns. More specifically, given a set of subpatterns $\mathbb{S}$ and a set of patterns $\mathbb{P}$, it computes what are the required subpatterns $\mathbb{M}$. This algorithm works in two phases: first, subpatterns are simulated to extract traces; second, for the given patterns, all sub-simulations are explored and checked against the ones from subpatterns. $\mathcal{S}_0$, $\delta_{\mathcal{S}}$, and $F_{\mathcal{S}}$ are respectively the initial state, a subset of transitions and finale states of automaton $\mathcal{S}$. The algorithm uses the same notation for $\mathcal{P}$. To help with the clarity of the algorithm, in addition to δ the set of transitions, we also consider two other set of transition. Loop contains transitions modeling repetitions. And Sink contains transitions skipping alternations.

It should also be noted that in practice, indexing stricter traces from $\mathbb{S}$ as well as relaxing the exploration of $\mathbb{P}$ have no impact on validity. This flexibility provides a way to incrementally improve the implementation while maintaining valid results.

5.2 Scalable Pattern Generation

As the core of TSQ patterns (*i.e.,* s-expressions) were originally used to compare syntax trees, they offer a direct isomorphism with nodes. This greatly facilitates synthesizing queries from code examples, as opposed to relational type queries which would require rebuilding nested node sequences. We also found that the simplicity of the syntax also enabled automating pattern processing and transformations.

To illustrate the objectives of our code pattern generation, we can use the `try fail catch` pattern motivating our overall approach. For example, the subpatterns presented in Figure 2b can be obtained straightforwardly from the pattern in Figure 2a represented as a syntax tree. The first, subpattern can be obtained by removing named nodes from the root named node, while the second subpattern can be obtained by simply extracting the corresponding named node. However, the specialized patterns presented in Figure 2c cannot be easily obtained from Figure 2a without access to language specific knowledge, similarly to relational queries. Thus, we leverage our code search capability to find instances of the initial pattern, then synthesize patterns corresponding to these instances, and finally proceed to iteratively remove named nodes and other syntax elements.

Instead of directly processing the TSQ syntax tree, we could also have used the TSQ IR, based on the automaton definition presented in Section 3.1. There generalizing a pattern would symmetrically be done by removing constraints on transitions (until reaching ϵ transitions). However, lacking a decompiler, we chose to generalize patterns using the TSQ syntax trees. Moreover, we were also able to leverage the HyperAST to efficiently represent large numbers of similar TSQ syntax trees. Indeed, as more and more patterns are obtained from iteratively removing syntax elements, we reach a stable point where generalizations produce already found patterns. Furthermore, given that we only remove syntax nodes, we easily parallelize the pattern generation, processing patterns by height and in parallel in each level, as new patterns are strictly smaller.

To control the pattern generation, we also propose to leverage TS Queries as a kind of meta-language. For the pattern synthesis, it enables filtering serialized patterns (*e.g.,* comments), choosing which gets their label serialized as

a positional predicate (*e.g.,* ignoring variables types while showing catch variable types). For the iterative simplification of patterns, it enables selecting how syntax nodes are removed, notably if all matching nodes must be removed at once or if they must be removed one by one (leading to a potential combinatorial growth).

5.2.1 Organizing and Ranking Generated Patterns.

Based on the pattern inclusion relation, we effectively mined while iteratively simplifying patterns, we can represent patterns as a post-order set. However, scaling to large numbers of generated patterns, transitive relations become superfluous (*i.e.,* it is noise from the generation process). Thus, we remove transitive relations to represent patterns in a lattice. With many patterns organized in a lattice, using techniques from Formal Concept Analysis (FCA), we are able to effectively compute metrics and rank top level pattern by coverage (over synthesized patterns). We also use our code search to gather the numbers of matching instances in few versions. Furthermore, we render pattern lattices, differently depending on the number of patterns. Small lattices can easily be by displayed graphically with all intermediate patterns, while large lattices, must be displayed more sparingly. Consequently, we show large lattices interactively, starting with all initial code instances and highest ranked patterns until all initial code instances are covered, then users can remove patterns they deem to general. Once they found some satisfying patterns, users can feed them back into the code search to observe how the number of found instances evolve over time (*i.e.,* commits).

6 Evaluation

6.1 Dataset

We selected a dataset of 18 well known open-source Java projects, which includes the projects used in the evaluation of HyperAST [17]. In the paper, we only show the detailed results obtained from Hadoop as it is large project and contains many occurrences of `try fail catch`. The other detailed results are available in our replication package [27].

6.2 Experimental Protocol

All experiments were executed on a 13-inch MacBook M1 Pro from 2020 with 16GB of RAM. We measure the efficiency of our incremental code search on a single core using both runtime latency and the number of nodes accessed per search per commit. We present our results using different types of plots. For the box plots, we employ two configurations for the whiskers: the usual `1.5*IRQ`, which we show with caps, and one where we extend the whisker from min to max which we show without caps. We provide a comprehensive replication package with the source code implementing our approach, the benchmarks and use cases presented in the evaluation, the intermediate results, and the scripts used to produce plots [27].

6.3 Incremental Search: scalability leveraging precomputed sub-patterns

We investigated two major dimensions: the syntax tree representation and the precomputed subpatterns. Regarding technical considerations, the tree representations of TS and HyperAST differ considerably. In particular, while TS trees are produced per file, HyperAST was designed for ASTs of entire commits and parses each unique file once. While TS represents Tree nodes uniquely, HyperAST deduplicates identical subtrees, thus we expect more fragmentation and fewer sequential reads in the HyperAST. Finally, the choice of precomputed subpatterns is expected to have an impact on performance, so comparing different combinations should allow us to investigate how the performance might vary, such as between projects of different sizes, or with different numbers and distributions of matching instances.

To ensure fair comparisons of search tasks, our benchmark adds a cache for TS parse trees per file (append-only), while also precomputing the subpatterns on the cached TS trees. Thus, we can also observe the impact of our subtree skipping technique in a more usual file-based approach, generalizing our findings outside syntax-level indexing.

To leverage the capabilities of the HyperAST and explore new ways of searching code, the searches presented in this section are all done on entire commits. Commits are processed consecutively–in a `git blame` fashion. We did not parallelize the search as it could impact the accuracy of our investigation into efficiency; this is due to possible issues related to OS level resource allocation, thermal throttling, and heterogeneous cores. In a practical production setting, for all the configurations we present in this evaluation, as the search process consists mostly of reads, parallelizing the searches at the file or commit level would divide the search time by however many core or threads can be allocated.

Figure 5 shows cumulative search time against number of results for different precomputed subpatterns and tree representations. To simulate an hypothetical search process of a developer using our approach, we start with the `try fail catch` pattern presented in Figure 2a, searching occurrences in the last 100 commits, then modifying the pattern (*i.e.,* Figure 2c) and searching the same 100 commits, then repeating this process 3 other times. The aforementioned simulation depicted in Figure 5 demonstrates the scalability of our approach to large projects, something that can be challenging for prior approaches.

RQ1: To what extent does our approach effectively find instances of patterns and their variants in large software repositories?

In Figure 5, the top facet presents a high level output of the search process, more specifically the number of matching occurrences (*i.e.,* numbers in bold for each query), then the addition and deletion of matched code instances (*i.e.,* the +X and -X). With this representation it is already possible

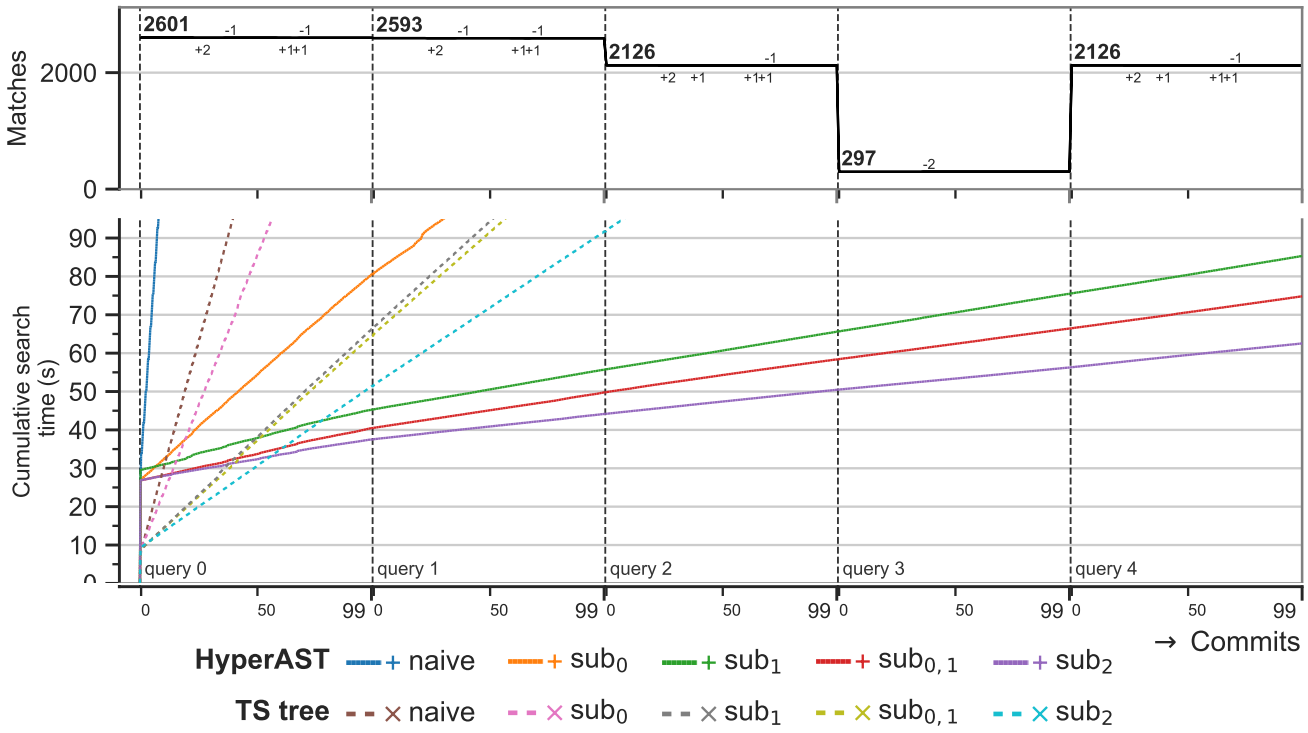
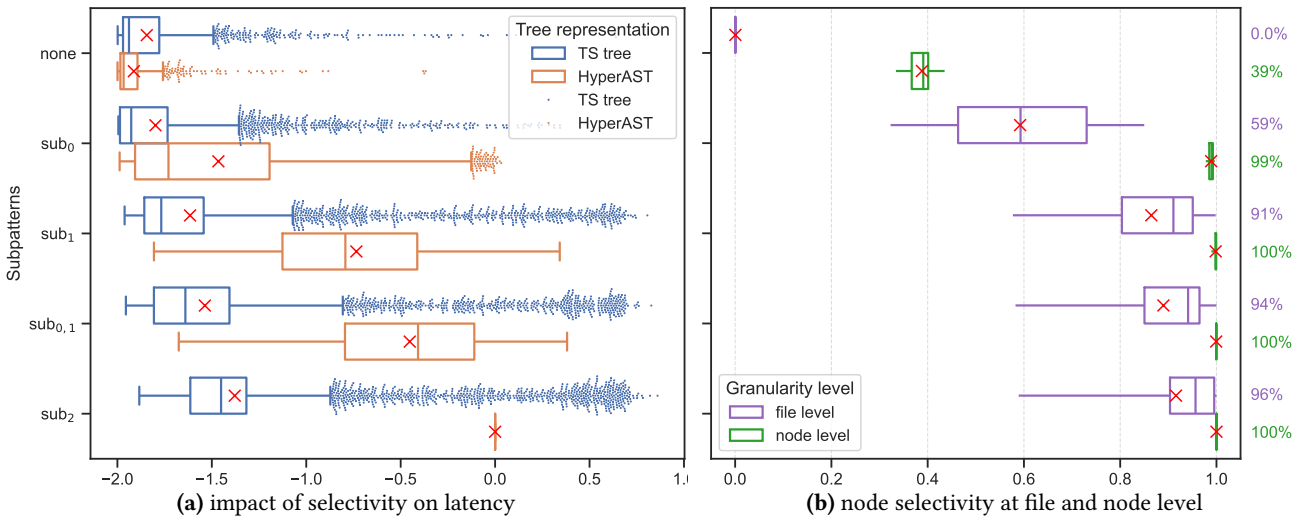


Figure 5. Searching variants of `try fail catch` (Figure 1c) in *Apache Hadoop* starting at `d5e97fe4`, over 100 commits. After the initial pattern, four other queries are sequentially executed on the same 100 commits (see Figure 2c). The top facet shows the number of matching nodes founds per commit. The bottom facet shows the cumulative time taken to do the search. Ten search configurations are shown over two dimension: supported by either HyperAST or TreeSitter representations; and with different precomputed subpatterns (see Figure 2b). In order queries represent: 0) the base pattern (Figure 3), 1) the call fail must be the last statement (first in Figure 2c), 2) fail must be preceded by at least one statement (second in Figure 2c), 3) at least 2, 4) exactly one. Further, details and plots, for other projects, can be found in the replication package [27].



(a) impact of selectivity on latency **(b)** node selectivity at file and node level

Figure 6. Selectivity of search for simplest `try fail catch` relative to precomputed subpattern with optimal selectivity (*i.e.*, the searched pattern is identical to the precomputed subpattern)

to get an idea of how common a given code pattern is. It also enables to notice general trends removing or adding code instances following the specifically observed code patterns. This representation also allowed us to evaluate the validity of our program as all implementations and optimizations gave us the same number of matches. We

found about 2600 instances of `try fail catch` in Hadoop, with the number slowly evolving throughout time. From the more specific patterns subsequently searched, we can deduce some interesting observations. For example, taking the changes at about 40 commits, for the base `try fail catch` pattern (query 0), we see a -1, but a -2 in query 3 which requires at least two statements before the call to fail. Thus, one `try fail catch` was removed and one changed by removing a statement.

The bottom facet shows the cumulative time taken for the search, concretely highlighting the duration of search for different precomputed subpatterns. First, it shows the impact of indexing the different subpatterns. Taking sub_2 which is exactly the pattern in `query 1`, it shows the maximum efficiency gains enabled by a maximally selective index. In 60 seconds, the sub_2 indexing enables searching 120 commits with TS Tree (*i.e.,* the contiguous tree) and 450 commits with HyperAST (*i.e.,* the DAG). At the opposite end of selectivity, the naive search (*i.e.,* without precomputed subpatterns) shows the worst possible selectivity, taking 90 seconds to process about 40 commits represented through TS Trees and only 10 commits with HyperAST. In the case of sub_0 (*i.e.,* indexing `(try_statement)`), we can clearly observe what should be the benefits of a finer grained indexing (*i.e.,* HyperAST), where the same 90 seconds enable processing only 50 commits (10 more) using TS Tree, but 120 commits with HyperAST (12 times the naive variant). Indexing sub_1 (*i.e.,* the call to fail) shows a more significant impact on runtime in both representations. TS Tree reaches a similar number of commits per second than sub_0 (almost parallel lines), while it improves to searching 500 commits in 85 seconds. Combining both subpatterns (*i.e.,* $\text{sub}_{0,1}$), we only observe a sizeable gain on the HyperAST representation, reducing the processing time of 500 commits by 10 seconds, notably only about as much time that separates it from sub_2. Besides, this runtime plot also shows drawbacks of the HyperAST variants, where an initial overhead can be observed to construct the representation and then execute the first search. Except for the naive search variant, it takes between 10 and 50 commits for the HyperAST representation to overcome the initial overhead. Consequently, without our approach it would be less practical to search code patterns on the HyperAST than directly on the TS Trees.

RQ2: To what extent does the granularity of files, as opposed to the AST level, influence the selectivity of abstract patterns?

Figures 6 and 6a presents the overall results of the incremental search benchmark, notably the influence of subpatterns on selectivity and the influence of selectivity on latency. Both show results relative to the optimal case (*i.e.,* sub2) which is identical to the initial searched pattern. All considered projects are included. Notably, Figure 6a show how our approach can leverage minimal precomputed subpatterns by increasing indexing granularity to syntax nodes.

Table 1. Results Pattern Generation on `try fail catch`

Name	#syn	#ex	prep (s)	#p	#cc	#top	gen (s)
hadoop	31M	1000	31.9	8746	460	927	129
flink	24M	1000	22.8	7509	456	894	90.4
gson	462K	232	0.404	1158	128	267	10.5
netty	5M	137	4.68	911	63	201	12.4
jackson-core	870K	127	0.694	1303	46	128	16.8
dubbo	3M	121	3.19	1052	87	213	16.5
quarkus	12M	111	11.8	871	63	213	13.6
spoon	2M	69	2.14	582	30	99	10.4
junit4	463K	28	0.391	499	10	95	5.41
jenkins	3M	30	3.09	271	17	31	5.34
logging-log4j2	3M	14	3.09	274	5	29	2.73
javaparser	2M	9	1.70	56	2	12	0.729
spark	1M	2	1.61	200	3	9	1.56
maven	1M	2	1.32	62	2	4	0.680
slf4j	231K	2	0.208	46	2	5	0.552
skywalking	1M	1	1.48	32	2	4	0.346
fastjson	3M	1	2.39	15	2	1	0.278

Incremental-search performances insights. With sufficiently selective subpatterns, it is possible to effectively apply our approach at the file level directly on TS Trees after parsing. Otherwise, even less selective subpatterns (*e.g.,* `(try_stmt)`) can be leveraged by our approach when using the HyperAST representation.

6.4 Pattern synthesis: Practicality of Automated Pattern Synthesis

To evaluate our pattern generation technique, we investigated the time taken to generate patterns from a set of examples obtained using our code search on a single commit (limited to 1000 instances). We search for instances of `try fail catch`, to more easily validate our results. Contrary to the benchmark evaluating our code search, the pattern generation uses all available cores. Indeed, from a large number of examples, we want our generation of representative patterns to explore a large variability space, leveraging redundancy in patterns. Moreover, using our meta-query controlling the removal of syntax nodes on patterns, we are able to control the granularity of the exploration. Detailed results can be found in our replication package [27], including the meta-queries controlling the generation, the markdown files using mermaid to display the lattices and hyperlinks to navigate between pretty printed patterns and examples.

RQ3: How effectively can we generate abstract code patterns representative of a set of examples?

Table 1 presents the results of our pattern-generation benchmark. For each project, it shows the size of the codebase in terms of syntax nodes, the numbers of instances of `try fail catch` found, followed by the time it took to build the HyperAST, then the number of generated patterns, and the number of connected components the form, followed

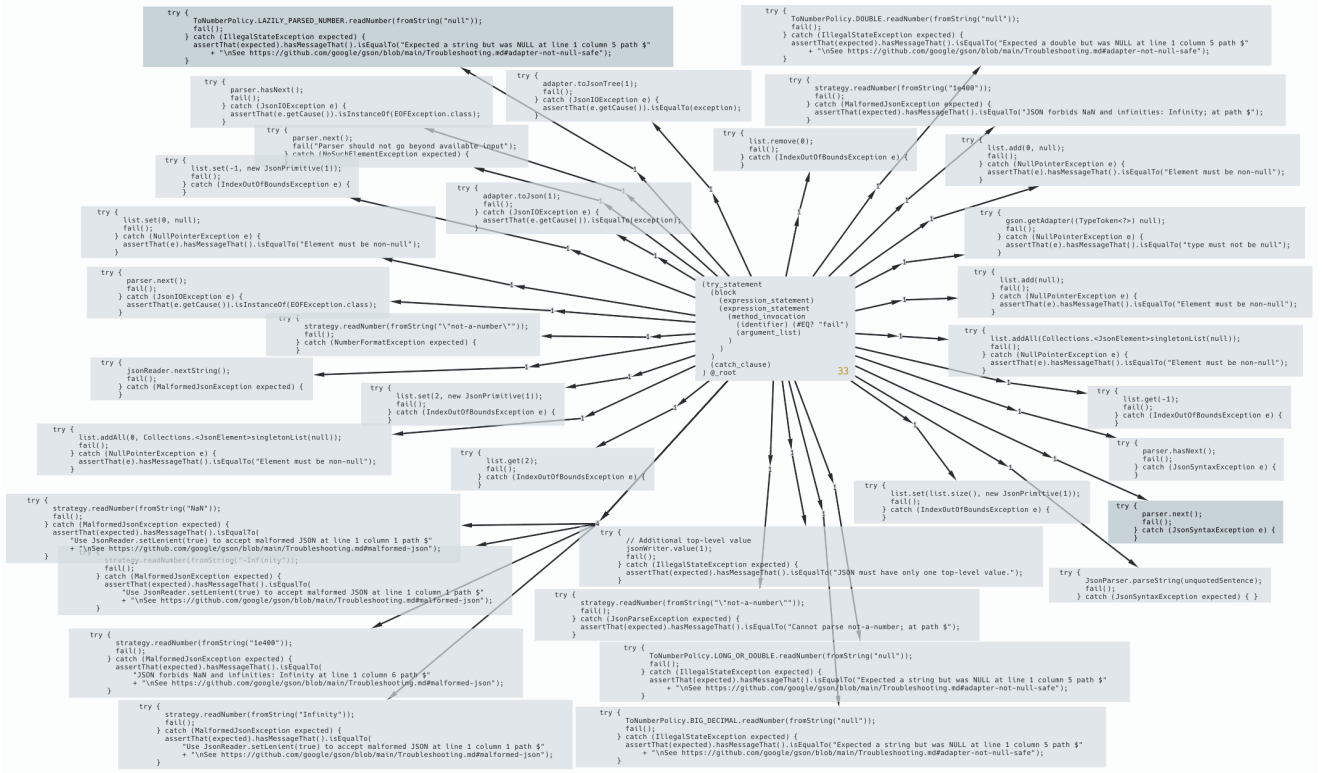


Figure 7. Interactive bigraph between examples (`try fail catch` instances part of a refactoring in commit 3d241ca of gson) and top patterns covering all of them (after we ignored two top patterns which were too general)

by the number of fully covering top-level patterns, and finally the time it took to generate and organize all the patterns. On the one hand, we observe a large range from 1 to 1000 in terms of examples, a range from 1 to 460 in terms of connected components, and top level patterns (1 to 927). On the other hand, in terms of preparation time, it varies from 208ms to 31.9s, while in terms of generation time, it varies from 278ms to 129s. We did not display the search time in the table, as it was taking too much time and did not give additional information (redundant with previous section). Nonetheless, finding instances of the searched pattern took between 123ms and 274ms. Additionally, we computed the correlations between our metrics using the Spearman correlation. We found that the number of syntax nodes was highly correlated to the preparation time (0.998 correlation with $p=2\times10^{-20}$). We did not find any correlation between search time and other metrics. Most importantly, we found a correlation of 0.958 between number of examples and the generation time ($p=5\times10^{-10}$), while we did not find a correlation between the number of syntax nodes and the generation time. Consequently, our pattern generation approach is capable of producing abstract code patterns representative of examples from entire codebases.

Complementarily to investigating runtime performance, we also show a screenshot of our interactive pattern-lattice exploration (see Figure 7). This pattern-lattice represents generated patterns concisely by only displaying the examples and the minimum number of top patterns necessary to cover all of them. Moreover, we have interactively hidden two patterns which were too general (*i.e.,* not showing details on the statement expressions). In the middle, we can observe the pattern resembling our example of `try fail catch`, with the number of examples covered (in the corner). Then, following the arrows materializing the iterative simplifications, we reach the pattern initially synthesized from the examples. One of those initial patterns represents 4 examples, due to the synthesis which ignored the content of string literals. Finally, following the arrows again we reach the examples.

7 Related Work

Multiple approaches exist to preprocess a code base for more complex analyzes and searches. They can be parsers or rely on external parsing and compiling libraries. Most of the approaches we found either target dialects of XPath, and SQL, or provide language specific syntaxic extension to provide code search capabilities.

CodeQL [1] is a relational database for AST nodes; it provides SQL-like query system with extensive language specific functions to support static analysis tasks. In addition, each programming language is integrated through so-called call extractors, which extract AST nodes from intermediate representations produced by existing compilers. Some of

these extractors rely on tree-sitter, notably Ruby and the QL language itself. CodeQL has a Java extractor based on a Kotlin compiler[5].

Google Code search [26] is one of the early code search tools deployed on a large software repository. To the best of our knowledge, it was the first industrial code search system providing regular expression queries with indexing. Chromium Code search[6] uses the open-sourced version.

GitHub Code Search [6] is analogous to Google Code Search in many ways. It is in part leveraging the Semantic library [13] and Tree-Sitter Queries to extract symbols of global declarations (*e.g.,* `symbol:Node`[7]), but it is a closed-source tool. They notably reported multiple technical improvements which enabled them to scale the index on multiple servers, giving code search capabilities over a large proportion of code in all open-sources projects on their platform.

Semantic Code Search [16] enables code search through natural language queries. It exhibits a low barrier to entry without having to learn a new language, but offers less precise control over the output of the search. Alternatively, this approach could be used for the initial search, then continuing with our approach to refine the results.

Spoon [22] is a code analysis library post-processing the output of the JDT. It provides many extensible interfaces and ways of augmenting nodes[8], internal DSLs, reference resolutions, and additional code analyses modules. Most notably for code search, Spoon provides XPath-like queries[9] which can also be synthesized from code elements.

JavaParser [15] is a parser which provides advanced reference analysis features, like Spoon it relies on a visitor interface to explore parsed ASTs.

Antlr is a very well established parser. It also provides a pattern syntax based on a subset of XPath[10].

SrcML is an AST analysis platform to search and manipulate source code using XML as intermediate representation. SrcML also provides a search interface based on XPath [25]. Similar to CodeQl, Bartman *et al.* have provided an abstraction extending SQL for code search [2]. After some initial measurements on Hadoop on the Mac M1, we found that it took about 115 seconds to initialize the XML representation and 7 seconds per query.

C and C++ possess multiple search and indexing tools, such as ctags and cscope. Alternatively, Coccinelle is a program analysis and transformation tool used in the Linux kernel, which can also be used to search code. Coccinelle extends the C syntax similarly to a patch and with additional annotations to abstract the otherwise concrete query. For example, it can specify that in `foo(bar)`, `bar` is an expression, such that, it would match `foo(42)`, `foo("baz")`, or `foo(x + 1)`.

[5]https://github.com/github/codeql/tree/main/java
[6]https://source.chromium.org/
[7]https://github.com/search?q=symbol:Node+language:Java&type=code
[8]spoon:/src/**/declaration/CtElementImpl.java#L108
[9]spoon:/src/main/java/spoon/reflect/path
[10]antlr.org/api/Java/org/antlr/v4/runtime/tree/xpath/XPath.html

8 Conclusion

In this paper, we presented our approach enabling the precise, efficient, and practical search of code patterns in large software histories. To properly present essential aspects of our approach, we proposed the formalization of Tree-Sitter Queries, which notably model the execution of a query as a Tree Walking Automaton. To make the code search precise, we adapted the Tree-Sitter Query system, leveraging its simple symbolic-expression syntax and language specific concepts extracted from Tree-Sitter grammars to search for abstract code patterns representing tests on exceptions, specifically `try fail catch`. To compensate for the usability limitations of pattern-based approaches, we proposed a technique to generate code patterns from examples, creating a feedback loop with our pattern-based code search. Finally, we also showed how we combined the patterns of Tree-Sitter and the DAG representation of HyperAST, to make both the search and the generation of code patterns efficient and scalable, finding abstract patterns and instances of `try fail catch` in hundreds of commits in the 20 open source projects that we considered.

In future work, we aim to lower the first query latency, by investigating how to directly query the tree-sitter output during DAG construction. We also want to lower the initial memory footprint, by improving the back-end DAG storage, or compress the subtrees. To scale horizontally, we could shard sub-patterns across multiple servers and dispatch queries based on maximal efficiency estimate.

Our pattern-based indexing could be used in the context of the Software Heritage archive to enable code searches without requiring exhaustive traversals, by providing additional metadata in the compressed graph [3, 4]. Then, leveraging the rest of our approach, it should be possible to establish datasets and training sets to evaluate and train coding models, iteratively refining the considered code patterns.

References

[1] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-oriented Queries on Relational Data. In *30th European Conference on Object-Oriented Programming (ECOOP 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:25. https://doi.org/10.4230/LIPIcs.ECOOP.2016.2

[2] Brian Bartman, Christian D Newman, Michael L Collard, and Jonathan I Maletic. 2017. srcQL: A syntax-aware query language for source code. In *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 467–471.

[3] Paolo Boldi, Antoine Pietri, Sebastiano Vigna, and Stefano Zacchiroli. 2020. Ultra-large-scale repository analysis via graph compression. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 184–194.

[4] Paolo Boldi and Sebastiano Vigna. 2004. The webgraph framework I: compression techniques. In *Proceedings of the 13th international conference on World Wide Web*. 595–602.

[5] Joel Brandt, Philip J Guo, Joel Lewenstein, Mira Dontcheva, and Scott R Klemmer. 2009. Two studies of opportunistic programming: interleaving web foraging, learning, and writing code. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 1589–1598.

[6] Scott Chacon. 2008. *GitHub Code Search*. GitHub Blog. https://github.blog/news-insights/the-library/github-code-search/ Updated January 4, 2019.

[7] Jérôme Champavère, Rémi Gilleron, Aurélien Lemay, and Joachim Niehren. 2008. Efficient inclusion checking for deterministic tree automata and DTDs. In *International Conference on Language and Automata Theory and Applications*. Springer, 184–195.

[8] Douglas A Creager and Hendrik van Antwerpen. 2022. Stack graphs: Name resolution at scale. *arXiv preprint arXiv:2211.01224* (2022).

[9] Luca Di Grazia and Michael Pradel. 2023. Code search: A survey of techniques for finding code. *Comput. Surveys* 55, 11 (2023), 1–31.

[10] Joost Engelfriet and Hendrik Jan Hoogeboom. 1999. Tree-walking pebble automata. In *Jewels are Forever: Contributions on Theoretical Computer Science in Honor of Arto Salomaa*. Springer, 72–83.

[11] Jean-Remy Falleri and Matias Martinez. 2024. Fine-grained, accurate and scalable source differencing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[12] Sébastien Ferré. 2014. *Reconciling Expressivity and Usability in Information Access*. Habilitation à diriger des recherches. Université de Rennes 1. https://inria.hal.science/tel-01100292

[13] Github. 2022. Semantic. https://github.com/github/semantic. Accessed: 2025-10-20.

[14] Lukáš Holík, Ondřej Lengál, Juraj Major, Adéla Štěpková, and Jan Strejček. 2025. On Complementation of Nondeterministic Finite Automata Without Full Determinization. In *International Symposium on Fundamentals of Computation Theory*. Springer, 221–237.

[15] Roya Hosseini and Peter Brusilovsky. 2013. Javaparser: A fine-grain concept indexing tool for java problems. In *CEUR Workshop Proceedings*, Vol. 1009. University of Pittsburgh, 60–63.

[16] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[17] Quentin Le Dilavrec, Djamel Eddine Khelladi, Arnaud Blouin, and Jean-Marc Jézéquel. 2022. HyperAST: Enabling Efficient Analysis of Software Histories at Scale. In *37th IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM, 1–12.

[18] Michael Martin, Benjamin Livshits, and Monica S Lam. 2005. Finding application errors and security flaws using PQL: a program query language. *Acm Sigplan Notices* 40, 10 (2005), 365–383.

[19] Luana Martins, Taher A Ghaleb, Heitor Costa, and Ivan Machado. 2024. A comprehensive catalog of refactoring strategies to handle test smells in Java-based systems. *Software Quality Journal* 32, 2 (2024), 641–679.

[20] Microsoft. 2022. Tree-Sitter. https://tree-sitter.github.io/tree-sitter/. Accessed: 2022-05-06.

[21] Meiyappan Nagappan, Romain Robbes, Yasutaka Kamei, Éric Tanter, Shane McIntosh, Audris Mockus, and Ahmed E Hassan. 2015. An empirical study of goto in C code from GitHub repositories. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 404–414.

[22] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. 2015. Spoon: A Library for Implementing Analyses and Transformations of Java Source Code. *Software: Practice and Experience* 46 (2015), 1155–1179.

[23] Caitlin Sadowski, Kathryn T Stolee, and Sebastian Elbaum. 2015. How developers search for code: a case study. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 191–201.

[24] Elvys Soares, Márcio Ribeiro, Rohit Gheyi, Guilherme Amaral, and André Santos. 2022. Refactoring test smells with junit 5: Why should developers keep up-to-date? *IEEE Transactions on Software Engineering* 49, 3 (2022), 1152–1170.

[25] Jan Stoklasa and Pavel Loupal. 2009. The XML-λ XPath Processor: Benchmarking and Results. In *International XML Database Symposium*. Springer, 53–66.

[26] Titus Winters, Tom Manshreck, and Hyrum Wright. 2020. *Software engineering at Google: Lessons learned from programming over time.* " O'Reilly Media, Inc.".

[27] X. 2026. *Searching Code Patterns in Large Software Histories: An Incremental Query System for Structural Patterns and their Variants*. https://doi.org/xx.xxxx/zenodo.xxxxxxxx